

Lezione 7 - Memoria di massa

FILES IN C

◆ Modello di accesso (stream)

Quando lavoriamo con i file in C, il contenuto NON viene caricato tutto in memoria.

👉 invece:

- chiediamo al sistema operativo di aprire il file
- otteniamo una struttura che rappresenta il file

Questa struttura contiene:

- quale file abbiamo aperto
- la modalità di accesso (lettura, scrittura, ecc.)
- la **posizione corrente** nel file (gestita dal sistema)

👉 questo ci permette di accedere al file come se fosse:

✳️ uno **stream** (flusso) di byte/caratteri

◆ Posizione corrente nel file

Immaginiamo un file con contenuto:

H E L L O _ W O R L D

👉 il sistema mantiene una **posizione corrente**

- le operazioni di lettura e scrittura avvengono su questa posizione
- dopo ogni operazione, la posizione viene aggiornata automaticamente

Esempio:

- posizione su 0

- leggiamo → otteniamo 0
- la posizione si sposta al carattere successivo

👉 quindi:

✳️ ogni operazione lavora sulla **posizione corrente** e la aggiorna

◆ Apertura dei file



Codice:

```
FILE * fopen(char * path, char * mode)
```

👉 parametri:

- path → percorso del file (es: "test.txt")
- mode → modalità di accesso

👉 modalità principali:

- "r" → lettura
- "w" → scrittura (sovrascrive il file)
- "a" → append (scrive alla fine del file)
- "r+" , "w+" → lettura e scrittura
- "rb" , "wb" , "ab" , "r+b" , "w+b" → modalità binaria

👉 ritorno:

- restituisce un puntatore a FILE
- rappresenta il file aperto

✳️ non serve conoscere l'implementazione interna

◆ Chiusura del file



Codice:

```
fclose(FILE * fp)
```

👉 serve per:

- liberare le risorse associate al file
- garantire che i buffer vengano svuotati (scrittura su disco)

👉 quindi:

✳ è fondamentale **chiudere sempre il file**



LETTURA E SCRITTURA (FILES IN C)

◆ Lettura e scrittura con fprintf e fscanf

Le funzioni `printf` e `scanf` hanno una versione che lavora sui file.

👉 invece di usare input/output standard:

- lavoriamo direttamente su un file



Codice:

```
fprintf(FILE * fp, char * format, ...)  
fscanf(FILE * fp, char * format, ...)
```

👉 funzionamento:

- **fprintf** → scrive su file (come `printf`)
- **fscanf** → legge da file (come `scanf`)

👉 osservazioni:

- possiamo usare `stdout` con `fprintf` per avere lo stesso comportamento di `printf`
- `fscanf` restituisce:
 - il numero di elementi letti

- EOF se il file è terminato

✳️ sono funzioni di *alto livello* perché gestiscono il formato

◆ Esempio di scrittura con fprintf



Codice:

```
#include <stdio.h>

int main(int argc, char * argv[])
{
    FILE * fp = fopen("test.txt", "w");
    for (int i = 0; i < 100; i++)
    {
        fprintf(fp, "i = %d\n", i);
    }
    fclose(fp);
    return 0;
}
```

👉 spiegazione del codice:

- apriamo il file "test.txt" in modalità scrittura con `fopen`
- con il `for` scorriamo i valori da 0 a 99
- ad ogni iterazione usiamo `fprintf` per scrivere una riga nel file nel formato `i = numero`
- alla fine chiudiamo il file con `fclose`

✳️ il file contiene **testo formattato**

◆ Esempio di lettura con fscanf



Codice:

```
#include <stdio.h>

void read_with_fscanf()
{
    FILE * fp = fopen("test.txt", "r");
    if (fp == NULL) {
        printf("Unable to open the file\n");
        return;
    }
    int x;
    while (fscanf(fp, "i = %d\n", &x) != EOF)
    {
        printf("%d\n", x);
    }
    fclose(fp);
}
```

👉 spiegazione del codice:

- apriamo il file "test.txt" in modalità lettura
- controlliamo se fopen ha restituito NULL , cioè se il file non è stato aperto
- dichiariamo una variabile x dove salvare il numero letto
- con fscanf leggiamo dal file seguendo il formato "i = %d\n"
- il ciclo continua finché non si arriva a EOF
- ogni numero letto viene stampato a video con printf
- alla fine chiudiamo il file

✳️ fscanf legge seguendo un **formato preciso**

◆ Lettura e scrittura a basso livello: getc e putc

👉 esistono funzioni più semplici che lavorano su singoli caratteri



Codice:

```
int getc(FILE * fp)
int putc(int c, FILE * fp)
```

👉 funzionamento:

- **getc** → legge un carattere dal file
- **putc** → scrive un carattere nel file

👉 ritorno:

- **getc** restituisce un carattere oppure EOF
- **putc** restituisce il carattere scritto oppure EOF in caso di errore

◆ Esempio con getc e putc



Codice:

```
void read_with_getc()
{
    FILE * fp = fopen("test.txt", "r");
    if (fp == NULL)
    {
        printf("Unable to open the file\n");
        return;
    }
    int c;
    while ((c = getc(fp)) != EOF)
    {
        putc(c, stdout);
    }
    fclose(fp);
}
```

👉 spiegazione del codice:

- apriamo il file in lettura
- controlliamo che il file sia stato aperto correttamente

- dichiariamo `c` come `int`, perché `getc` può restituire anche `EOF`
- leggiamo un carattere alla volta con `getc`
- ogni carattere letto viene scritto su `stdout` con `putc`
- continuiamo finché non raggiungiamo `EOF`
- alla fine chiudiamo il file

✳ qui lavoriamo **carattere per carattere**

◆ Differenza tra alto livello e basso livello

👉 `fprintf` / `fscanf` :

- lavorano con formati
- sono più comode
- fanno operazioni più “ricche”

👉 `getc` / `putc` :

- lavorano su singoli byte/caratteri
- sono più semplici
- richiedono una chiamata per ogni carattere

✳ quindi `getc` e `putc` sono interfacce più *basse* rispetto a `fprintf` e `fscanf`

📌 LETTURA E SCRITTURA A BLOCCHI

◆ Idea

Le funzioni viste prima lavorano:

- su testo (`fprintf` / `fscanf`)
- su singoli caratteri (`getc` / `putc`)

👉 ma spesso vogliamo lavorare su **blocchi di dati**

👉 ad esempio:

- array di interi
- array di float

- strutture dati

✳ leggere/scrivere un elemento alla volta è inefficiente

◆ fread

👉 nota:

- se vengono letti meno elementi del previsto:
 - può essere fine file
 - oppure errore
- possiamo usare:

```
feof(FILE * fp)
```



Codice:

```
size_t fread(void * ptr, size_t size, size_t nmemb, FILE * fp)
```

👉 parametri:

- ptr → puntatore dove salvare i dati letti
- size → dimensione di ogni elemento
- nmemb → numero di elementi da leggere
- fp → file da cui leggere

👉 ritorno:

- numero di elementi effettivamente letti

👉 osservazione:

- può essere minore di nmemb se:
 - siamo arrivati a fine file
 - oppure c'è stato un errore

✳ legge **più elementi in una sola operazione**

◆ fwrite



Codice:

```
size_t fwrite(void * ptr, size_t size, size_t nmemb, FILE * fp)
```

👉 parametri:

- ptr → puntatore ai dati da scrivere
- size → dimensione di ogni elemento
- nmemb → numero di elementi da scrivere
- fp → file su cui scrivere

👉 ritorno:

- numero di elementi effettivamente scritti

✳ scrive **blocchi di dati in modo efficiente**

◆ Osservazione importante

👉 rispetto a getc e putc :

- non lavoriamo più su un carattere alla volta
- lavoriamo su **blocchi interi di memoria**

✳ molto più efficiente per grandi quantità di dati

◆ Esempio: scrittura e lettura di un array



Codice:

```
#include <stdio.h>

void write_array(int * v, int len)
{
    FILE * fp = fopen("test.bin", "wb");
    fwrite(v, sizeof(int), len, fp);
    fclose(fp);
}

int read_array(int * v, int len)
{
    FILE * fp = fopen("test.bin", "rb");
    if (fp == NULL) {
        printf("Unable to open the file\n");
        return 0;
    }
    int bytes_read = fread(v, sizeof(int), len, fp);
    fclose(fp);
    return bytes_read;
}

int main(int argc, char * argv[])
{
    int v[5] = {10, 20, 40, 50, -5};
    write_array(v, 5);

    int w[5] = {0, 0, 0, 0, 0};
    int n = read_array(w, 5);

    printf("%d integers read\n", n);

    for (int i = 0; i < 5; i++)
    {
        printf("%d ", w[i]);
    }

    printf("\n");
}
```

```
    return 0;
}
```

👉 spiegazione del codice:

- `write_array` :
 - apre il file in modalità **binaria scrittura** ("wb")
 - scrive tutto l'array in una sola chiamata a `fwrite`
 - chiude il file
- `read_array` :
 - apre il file in modalità **binaria lettura** ("rb")
 - controlla che il file esista
 - legge gli elementi con `fread`
 - ritorna quanti elementi sono stati letti
- `main` :
 - crea un array `v`
 - lo salva su file
 - legge i dati in `w`
 - stampa quanti valori sono stati letti
 - stampa i valori

✳ i dati vengono salvati in **formato binario**, non come testo

◆ Differenza tra file di testo e file binari

👉 file di testo:

- leggibili da umano
- più facili da debug
- meno efficienti

👉 file binari:

- non leggibili direttamente
- più compatti
- più veloci da leggere/scrivere

✳ `fread` e `fwrite` sono pensate per **file binari**

◆ Idea importante

👉 con `fread` e `fwrite`:

- lavoriamo direttamente sulla memoria
- evitiamo parsing di testo
- riduciamo il numero di operazioni

✳️ fondamentale per le **prestazioni**

📌 MUOVERSI ALL'INTERNO DI UN FILE

◆ Idea

Finora abbiamo visto accessi **sequenziali** ai file:

👉 leggiamo dall'inizio e andiamo avanti

👉 ma in molti casi questo **NON** è efficiente

Esempio:

- array ordinato su file
- vogliamo accedere a una posizione specifica

✳️ leggere tutto fino a quel punto sarebbe troppo costoso

◆ Accesso non sequenziale

👉 possiamo spostarci direttamente in un punto del file

👉 questo si fa con:

💻 **Codice:**

```
int fseek(FILE * fd, long offset, int whence)
```

◆ Parametri di fseek

👉 fd :

- file su cui operare

👉 offset :

- quanto spostarsi (in **byte**)
- può essere anche negativo

👉 whence :

- punto di riferimento per lo spostamento

◆ Valori di whence

- SEEK_SET → dall'inizio del file
- SEEK_CUR → dalla posizione corrente
- SEEK_END → dalla fine del file

👉 esempio:

- fseek(fp, 0, SEEK_SET) → vai all'inizio
- fseek(fp, 0, SEEK_END) → vai alla fine

◆ Valore di ritorno

👉 fseek restituisce:

- 0 → successo
- valore diverso → errore

◆ ftell

👉 per sapere dove siamo nel file:



Codice:

```
long ftell(FILE * fp)
```

👉 ritorna:

- posizione corrente nel file (in byte)
- ✳️ utile per capire dove ci troviamo durante lettura/scrittura

◆ Esempio: accesso casuale nel file



Codice:

```
#include <stdio.h>
#include <stdlib.h>

void create_file(int n)
{
    FILE * fp = fopen("test.bin", "wb");
    for (int i = 0; i < n; i++) {
        fwrite(&i, sizeof(int), 1, fp);
    }
    fclose(fp);
}

int main(int argc, char * argv[])
{
    const int n = 1000;
    create_file(n);

    FILE * fp = fopen("test.bin", "rb");

    for (int i = 0; i < 10; i++) {
        int pos = sizeof(int) * (rand() % n);
        fseek(fp, pos, SEEK_SET);

        int val;
        fread(&val, sizeof(int), 1, fp);

        printf("Letto %d in posizione %d\n", val, pos);
    }

    fclose(fp);
    return 0;
}
```

👉 spiegazione del codice:

- creiamo un file binario con numeri da 0 a n-1
- apriamo il file in lettura
- generiamo posizioni casuali

- con `fseek` ci spostiamo direttamente in quella posizione
- leggiamo il valore con `fread`
- stampiamo il valore letto

✳️ accesso diretto senza scorrere tutto il file

◆ Osservazione importante

👉 con `fseek` :

- possiamo evitare accessi sequenziali
- possiamo fare accessi casuali

👉 ma:

✳️ ogni accesso su disco è costoso

◆ Idea importante

👉 rispetto alla memoria RAM:

- accessi casuali su disco sono molto più lenti

👉 quindi:

✳️ bisogna ridurre il numero di accessi

📌 SISTEMA OPERATIVO E MEMORIA DI MASSA

◆ Buffer e comportamento del sistema operativo

Quando lavoriamo con i file, il sistema operativo NON accede direttamente al disco ad ogni operazione.

👉 invece utilizza dei **buffer**

👉 in particolare:

- in scrittura:
 - i dati non vengono scritti subito su disco
 - vengono accumulati in un buffer
 - vengono scritti quando:
 - il buffer è pieno
 - oppure passa abbastanza tempo
- in lettura:
 - il sistema NON legge un byte alla volta
 - legge **blocchi di dati**
 - i dati vengono salvati in memoria principale

👉 accessi successivi allo stesso blocco:

- avvengono direttamente in memoria
- senza accedere al disco

✳️ questo meccanismo è chiamato **buffer cache**

◆ Effetto del buffer

👉 grazie al buffering:

- riduciamo il numero di accessi al disco
- miglioriamo le prestazioni
- sfruttiamo la memoria principale come cache

✳️ il sistema operativo ottimizza automaticamente gli accessi

◆ Problema: il costo delle operazioni

👉 normalmente, negli algoritmi:

- ogni operazione ha costo unitario

👉 ma questo NON è vero per la memoria di massa

👉 infatti:

- accesso alla CPU → velocissimo
- accesso alla RAM → veloce
- accesso al disco → lentissimo

✳ differenze di migliaia o milioni di volte

◆ Differenza enorme di tempi

👉 nelle slide viene mostrato un esempio:

- accesso cache L1 → ~100 ns
- accesso disco → può arrivare a millisecondi

👉 tradotto:

✳ se la cache fosse 1 secondo:

- la RAM sarebbe ore
- il disco sarebbe giorni

👉 quindi:

✳ la memoria di massa è estremamente lenta

◆ Conseguenza fondamentale

👉 non possiamo più ragionare come prima

👉 non basta contare le operazioni:

✳ dobbiamo minimizzare gli accessi al disco

◆ Idea chiave

👉 le prestazioni NON dipendono solo dall'algoritmo

👉 ma anche da:

- dove stanno i dati
- come vengono letti
- quante volte accediamo al disco

✳️ la memoria di massa cambia completamente il modo di analizzare gli algoritmi

📌 MODELLO DI COSTO E ARRAY SCAN

◆ Idea

Quando lavoriamo con la memoria di massa, non possiamo più usare il modello classico:

👉 ogni operazione = costo unitario

👉 perché:

✳️ gli accessi al disco sono estremamente costosi

◆ Nuovo modello (I/O model)

👉 introduciamo un nuovo modo di analizzare i costi

👉 consideriamo:

- N → numero totale di elementi
- B → dimensione di un blocco (numero di elementi letti insieme)
- M → dimensione della memoria interna

👉 ipotesi:

- i dati sono divisi in blocchi di dimensione B
- leggiamo sempre blocchi interi
- in memoria possiamo tenere al massimo M elementi

👉 quindi possiamo mantenere in memoria:

✳️ $\lfloor M / B \rfloor$ blocchi

◆ Ipotesi fondamentale

👉 assumiamo:

✳ $N \gg M \gg B$

cioè:

- i dati sono molto più grandi della memoria
- la memoria è molto più grande di un blocco

◆ Cosa contiamo

👉 nel nuovo modello:

- ogni operazione interna $\rightarrow$ costo 0
- ogni accesso al disco $\rightarrow$ costo 1

👉 quindi:

✳ contiamo solo le operazioni di I/O

◆ Array scan

👉 consideriamo un problema semplice:

- abbiamo un file con N interi
- vogliamo sommarli tutti

◆ Strategia

👉 leggiamo il file a blocchi di dimensione B

👉 quindi:

- ogni lettura prende B elementi
- servono circa N / B letture

👉 costo totale:

✳ $O(\lceil N / B \rceil)$

◆ Interpretazione

👉 invece di leggere elemento per elemento:

- leggiamo blocchi interi
- riduciamo il numero di accessi

✳ molto più efficiente

◆ Esempio (intuitivo)

👉 immaginiamo:

- blocchi da 4 elementi

file:

1 2 5 6 7 8 1 0 4 2 3 9

👉 diventa:

- blocco 1 → 1 2 5 6
- blocco 2 → 7 8 1 0
- blocco 3 → 4 2 3 9

👉 invece di 12 letture:

- facciamo 3 letture

✳ enorme risparmio

◆ Esempio reale: somma su file



Codice:

```
#include <stdio.h>
#include <stdlib.h>

void create_file(int n)
{
    FILE * fp = fopen("test.bin", "wb");
    for (int i = 0; i < n; i++) {
        fwrite(&i, sizeof(int), 1, fp);
    }
    fclose(fp);
}

int scan_array()
{
    FILE * fp = fopen("test.bin", "rb");
    int buffer[1024];
    int elem_read;
    int sum = 0;

    while ((elem_read = fread(buffer, sizeof(int), 1024, fp)) != 0) {
        for (int i = 0; i < elem_read; i++)
        {
            sum += buffer[i];
        }
    }

    fclose(fp);
    return sum;
}

int main(int argc, char * argv[])
{
    const int n = 1000000;
    create_file(n);

    int sum = scan_array();
    printf("Sum of the values: %d\n", sum);
}
```

```
    return 0;  
}
```

👉 spiegazione del codice:

- `create_file` :
 - crea un file binario con valori da 0 a $n-1$
 - scrive un intero alla volta con `fwrite`
- `scan_array` :
 - apre il file in lettura
 - usa un buffer di 1024 interi
 - legge blocchi con `fread`
 - somma gli elementi del buffer
 - continua finché `fread` restituisce valori
- ciclo principale:
 - legge blocchi invece di singoli elementi
 - riduce drasticamente gli accessi al disco

✳ implementazione reale del modello $O(N / B)$

◆ Idea fondamentale

👉 per essere efficienti:

- NON dobbiamo ridurre le operazioni CPU
- dobbiamo ridurre gli accessi al disco

👉 quindi:

✳ lavorare a blocchi è essenziale

📌 STRUTTURE AD ALBERO SU DISCO

◆ Idea

Consideriamo di memorizzare una struttura dati su memoria di massa.

👉 esempio:

- un **albero binario**
- utilizzato per ricerche (es. ricerca binaria)

◆ Problema

👉 supponiamo:

- l'albero contiene N elementi
- ogni nodo è memorizzato in un blocco

👉 per raggiungere una foglia:

- dobbiamo attraversare l'albero
- quindi fare circa:

✳ **$\log_2 N$ accessi al disco**

◆ Perché è un problema

👉 ogni accesso al disco è molto costoso

👉 quindi:

✳ anche pochi accessi diventano molto lenti

◆ Idea migliorativa

👉 invece di usare un albero binario:

- usiamo un albero con più figli (*fanout alto*)

👉 cioè:

- ogni nodo contiene più valori
- ogni nodo punta a più figli

◆ Effetto del fanout

👉 se ogni nodo ha k figli:

- altezza dell'albero = $\log_k N$

👉 quindi:

- meno livelli
- meno accessi al disco

◆ Esempio

👉 caso classico:

- $N = 10^9$

👉 albero binario:

- altezza ≈ 30
- $\rightarrow 30$ accessi al disco

👉 albero con $k = 1000$:

- altezza ≈ 3
- $\rightarrow$ solo 3 accessi

✳️ miglioramento enorme

◆ Idea fondamentale

👉 quando lavoriamo su disco:

- NON conta solo la struttura dati
- conta **quanti accessi facciamo**

👉 quindi:

✳️ meglio strutture con pochi livelli

◆ Collegamento importante

👉 questo concetto è alla base di:

- B-tree
- database
- filesystem

✳️ tutte strutture progettate per ridurre gli accessi al disco

📌 MMAP – MAPPATURA DI FILE IN MEMORIA

◆ Idea

Finora abbiamo lavorato con i file usando:

- fread / fwrite
- fseek

👉 cioè:

✳️ gestiamo manualmente lettura e scrittura

👉 con **mmap** cambia tutto:

👉 possiamo accedere al file come se fosse in memoria

✳️ come se fosse un array

◆ Obiettivo

👉 vogliamo:

- accedere ai dati con indici

- evitare fread / fwrite
- caricare solo ciò che serve

✳️ senza leggere tutto il file

◆ Memoria virtuale

👉 lo spazio degli indirizzi è molto più grande della memoria fisica

👉 quindi possiamo:

- “far finta” che tutto il file sia in memoria
- ma caricare solo le parti necessarie

✳️ questo è possibile grazie alla **memoria virtuale**

◆ Come funziona mmap

👉 il sistema operativo:

- associa il file a una zona di memoria virtuale
- restituisce un puntatore

👉 inizialmente:

- i dati NON sono in RAM
- esistono solo come mapping

◆ Accesso ai dati

👉 quando accediamo a una posizione:

- se i dati NON sono in memoria
→ il sistema operativo li carica dal disco
- se sono già presenti
→ accesso immediato

✳️ caricamento *on-demand*

◆ Pagine di memoria

👉 la memoria è divisa in **pagine**

👉 quando accediamo:

- viene caricata un'intera pagina
- non solo il dato richiesto

👉 quindi:

✳️ un accesso può caricare molti dati utili

◆ Modifica dei dati

👉 possiamo modificare i dati direttamente:

- scrivendo in memoria
- senza usare fwrite

👉 le modifiche:

- vengono propagate al file
- NON necessariamente subito

✳️ il sistema operativo decide quando scrivere

◆ Gestione della memoria

👉 se la memoria si riempie:

- pagine non modificate → eliminate
- pagine modificate → scritte su disco

✳️ il sistema gestisce automaticamente tutto

◆ Funzione mmap



Codice:

```
void * mmap(void * addr, size_t len, int prot, int flags, int fd, off_t offset)
```

◆ Parametri

👉 addr :

- indirizzo dove mappare (di solito 0 → decide il sistema)

👉 len :

- dimensione del mapping

👉 prot :

- permessi:
 - PROT_READ
 - PROT_WRITE
 - PROT_EXEC

👉 flags :

- tipo di mapping:
 - MAP_SHARED → modifiche visibili nel file
 - MAP_PRIVATE → modifiche locali

👉 fd :

- file descriptor

👉 offset :

- punto del file da cui iniziare (multiplo della pagina)

◆ Valore di ritorno

👉 mmap restituisce:

- puntatore alla memoria
- oppure MAP_FAILED in caso di errore

◆ Sequenza operativa

👉 per usare mmap:

1. aprire il file con `fopen`
2. ottenere il file descriptor con `fileno`
3. (opzionale) ottenere dimensione con `stat`
4. chiamare `mmap`

◆ Rimozione del mapping

👉 per liberare la memoria:



Codice:

```
munmap(void * addr, size_t size)
```

👉 inoltre:

- `fclose` chiude il file
- `munmap` rimuove il mapping

◆ Scrittura forzata

👉 le modifiche NON sono immediate

👉 possiamo forzarle con:



Codice:

```
msync(...)
```

✳️ utile per garantire scrittura su disco

◆ Funzioni utili

👉 `ftruncate` :

- cambia dimensione del file

👉 `madvise` :

- suggerisce al sistema come useremo la memoria

◆ Esempio mmap



Codice:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/mman.h>
#include <sys/stat.h>

int main(int argc, char * argv[])
{
    char * data;
    struct stat sbuf;
    FILE * fd = fopen("mmap_demo", "r+");
    int fdnum = fileno(fd);

    stat("mmap_demo", &sbuf);

    data = mmap((void *)0, sbuf.st_size, PROT_READ, MAP_SHARED, fdnum, 0);

    for (int i = 0; i < 10; i++)
    {
        printf("%c %c\n", data[i], getc(fd));
    }

    munmap(data, sbuf.st_size);
    fclose(fd);

    return 0;
}
```

👉 spiegazione del codice:

- apriamo il file
- otteniamo dimensione con `stat`
- mappiamo il file in memoria
- accediamo ai dati come array (`data[i]`)
- confrontiamo con accesso tradizionale (`getc`)
- liberiamo memoria con `munmap`
- chiudiamo il file

✳️ accesso diretto senza fread

◆ Vantaggi

👉 con mmap:

- accesso diretto come memoria
- meno codice di I/O
- possibile maggiore efficienza
- utile con più processi/thread

◆ Svantaggi

👉 mmap:

- può essere più complesso da usare
- non sempre più veloce (accessi sequenziali)
- dipende dal sistema operativo
- limitato dallo spazio di indirizzamento

👉 inoltre:

- il controllo sul caching è delegato al sistema

◆ Idea fondamentale

👉 mmap permette di:

- trattare file come memoria
- delegare la gestione al sistema operativo
- ridurre il codice di I/O

✳️ è un cambio di paradigma rispetto a fread/fwrite